A03-RAG-Patterns

1 RAG Patterns

Document Version: 1.0
Generated: December 4, 2025
Standard: Retrieval-Augmented Generation Patterns
Status: Emerging Standard

1.1 Overview

Retrieval-Augmented Generation (RAG) augments LLMs with external knowledge retrieval. RAG patterns provide:

- **Grounding**: Reduce hallucinations with factual context
- **Currency**: Access up-to-date information
- **Explainability**: Show source documents
- **Efficiency**: Smaller models with better results
- **Privacy**: Keep data local

1.1.1 Core RAG Components

1. **Chunking**: Split documents into retrievable units
2. **Embedding**: Convert text to vectors
3. **Retrieval**: Find relevant chunks
4. **Reranking**: Score and filter results
5. **Augmentation**: Inject context into prompt
6. **Generation**: LLM produces answer with context

1.2 Authoritative References

- **RAG Survey**: https://arxiv.org/abs/2312.10997
- **LangChain RAG**: https://docs.langchain.com/docs/use_cases/qa_structured_data
- **LlamaIndex**: https://gpt-index.readthedocs.io
- **Haystack**: https://docs.deepset.ai/latest

1.3 Format Structure

1.3.1 Chunking Strategies

1.3.1.1 Fixed-Size Chunking

Simple approach with size + overlap:

```python
def fixed_size_chunks(
    text: str,
    chunk_size: int = 512,
    overlap: int = 50
) -> list:
    """Split text into fixed-size chunks"""
    chunks = []
    start = 0

    while start < len(text):
        end = min(start + chunk_size, len(text))
        chunks.append(text[start:end])
        start = end - overlap  # Back up for overlap

    return chunks

# Example
text = "Long document text..." * 100
chunks = fixed_size_chunks(text, chunk_size=512, overlap=50)
print(f"Generated {len(chunks)} chunks")
```

Configuration:

```json
{
  "chunking_strategy": "fixed",
  "chunk_size": 512,
  "overlap": 50,
  "unit": "characters"
}
```

1.3.1.2 Semantic Chunking

Split on semantic boundaries:

```python
from sentence_transformers import SentenceTransformer
import numpy as np

def semantic_chunks(
    text: str,
    threshold: float = 0.5,
    model_name: str = "all-MiniLM-L6-v2"
) -> list:
    """Split text based on semantic similarity"""

    model = SentenceTransformer(model_name)
    sentences = text.split(". ")

    if len(sentences) < 2:
        return [text]

    # Embed sentences
    embeddings = model.encode(sentences)

    # Compute similarities
    chunks = [[sentences[0]]]

    for i in range(1, len(sentences)):
        # Similarity to last chunk's last sentence
        similarity = np.dot(
            embeddings[i],
            embeddings[chunks[-1][-1].__hash__()]
        )

        if similarity > threshold:
            chunks[-1].append(sentences[i])
        else:
            chunks.append([sentences[i]])

    # Join back
    return [". ".join(chunk) + "." for chunk in chunks]
```

1.3.1.3 Hierarchical Chunking

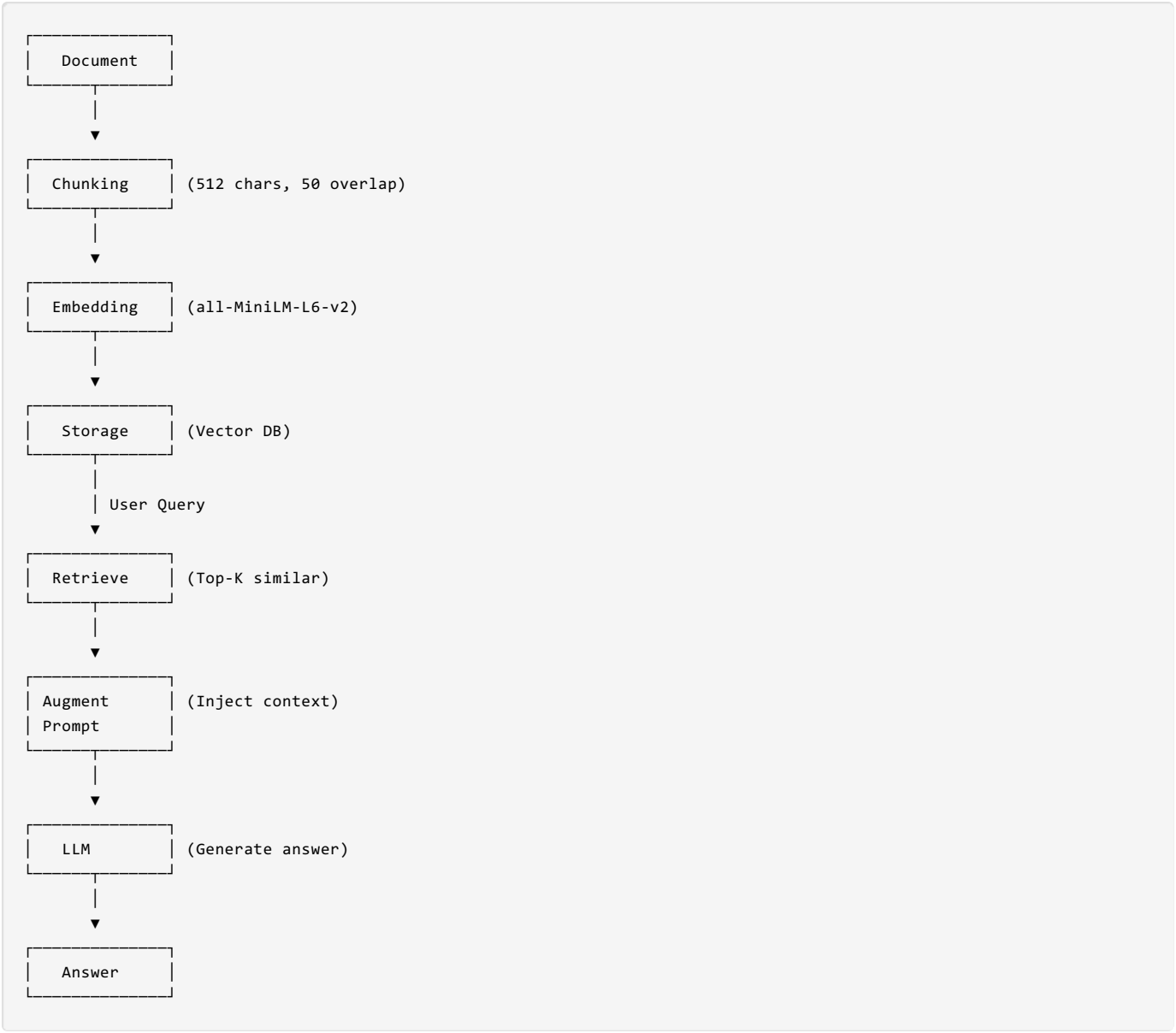
Tree structure for multi-level retrieval:

```python
class HierarchicalChunker:
    def __init__(self, chunk_size: int = 1024, sub_chunk_size: int = 256):
        self.chunk_size = chunk_size
        self.sub_chunk_size = sub_chunk_size

    def chunk(self, text: str) -> dict:
        """Create hierarchical chunks"""

        # Level 1: Large chunks
        chunks = []
        for i in range(0, len(text), self.chunk_size):
            chunk = text[i:i + self.chunk_size]

            # Level 2: Sub-chunks
            sub_chunks = []
            for j in range(0, len(chunk), self.sub_chunk_size):
                sub_chunk = chunk[j:j + self.sub_chunk_size]
                sub_chunks.append(sub_chunk)

            chunks.append({
                "level1": chunk,
                "level2": sub_chunks,
                "position": len(chunks)
            })

        return chunks
```

1.4 Procedural Use-Cases

1.4.1 Use-Case 1: Simple RAG Pipeline

Architecture Diagram:



Python Implementation:

```python
from sentence_transformers import SentenceTransformer
from pinecone import Pinecone
import anthropic

class SimpleRAG:
    def __init__(self, index_name: str):
        self.model = SentenceTransformer("all-MiniLM-L6-v2")
        self.pc = Pinecone(api_key="xxx")
        self.index = self.pc.Index(index_name)
        self.llm = anthropic.Anthropic()

    def query(self, question: str, top_k: int = 5) -> str:
        """Answer question using RAG"""

        # 1. Embed query
        query_embedding = self.model.encode(question)

        # 2. Retrieve context
        results = self.index.query(
            vector=query_embedding,
            top_k=top_k,
            include_metadata=True
        )

        context_docs = [
            match.metadata["text"]
            for match in results.matches
        ]

        # 3. Augment prompt
        context = "\n\n".join(context_docs)
        prompt = f"""Use the following context to answer the question.

Context:
{context}

Question: {question}

Answer:"""

        # 4. Generate
        response = self.llm.messages.create(
            model="claude-3-sonnet-20240229",
            max_tokens=1024,
            messages=[
                {"role": "user", "content": prompt}
            ]
        )

        return response.content[0].text

# Usage
rag = SimpleRAG("my-index")
answer = rag.query("What is the capital of France?")
print(answer)
```

1.4.2 Use-Case 2: RAG with Reranking

Reranking improves precision by rescoring retrieved documents:

```python
from sentence_transformers import CrossEncoder

class RAGWithReranking:
    def __init__(self, index_name: str):
        self.embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
        self.reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-12-v2")
        self.index = Pinecone().Index(index_name)
        self.llm = anthropic.Anthropic()

    def query(self, question: str, top_k: int = 5, rerank_k: int = 3) -> str:
        """RAG with reranking"""

        # 1. Initial retrieval (more results)
        query_embedding = self.embedding_model.encode(question)

        initial_results = self.index.query(
            vector=query_embedding,
            top_k=top_k * 2,  # Get more for reranking
            include_metadata=True
        )

        candidate_docs = [
            match.metadata["text"]
            for match in initial_results.matches
        ]

        # 2. Rerank
        reranked_scores = self.reranker.predict([
            [question, doc] for doc in candidate_docs
        ])

        # Get top reranked documents
        top_indices = np.argsort(reranked_scores)[-rerank_k:][::-1]
        context_docs = [candidate_docs[i] for i in top_indices]

        # 3. Generate answer with reranked context
        context = "\n\n".join(context_docs)
        prompt = f"""Based on this context, answer the question:

Context:
{context}

Question: {question}

Answer:"""

        response = self.llm.messages.create(
            model="claude-3-sonnet-20240229",
            max_tokens=1024,
            messages=[{"role": "user", "content": prompt}]
        )

        return response.content[0].text

# Usage
rag = RAGWithReranking("my-index")
answer = rag.query("What is machine learning?")
```

1.4.3 Use-Case 3: Agentic RAG (Multiple Queries)

Use tools to iteratively search and refine:

```python
from typing import Optional

class AgenticRAG:
    def __init__(self, index_name: str):
        self.index = Pinecone().Index(index_name)
```

```
        self.model = SentenceTransformer("all-MiniLM-L6-v2")
        self.llm = anthropic.Anthropic()

    def search(self, query: str, top_k: int = 5) -> list:
        """Search documents"""
        query_emb = self.model.encode(query)
        results = self.index.query(
            vector=query_emb,
            top_k=top_k,
            include_metadata=True
        )
        return [m.metadata["text"] for m in results.matches]

    def answer_with_search(self, question: str, max_searches: int = 3) -> str:
        """Answer by searching multiple times if needed"""

        search_results = []
        queries = [question]  # Initial query

        for i in range(max_searches):
            # Search
            for q in queries[i:i+1]:  # Current batch
                results = self.search(q, top_k=3)
                search_results.extend(results)

            # Check if we need more searches
            context = "\n\n".join(search_results[:3])

            check_prompt = f"""Given this context, can you fully answer the question "{question}"?

Context:
{context}

Answer "yes" if you have enough information, or list 2-3 follow-up searches to run."""

            response = self.llm.messages.create(
                model="claude-3-sonnet-20240229",
                max_tokens=200,
                messages=[{"role": "user", "content": check_prompt}]
            )

            response_text = response.content[0].text.lower()
            if "yes" in response_text:
                break
            else:
                # Extract suggested searches
                # (simplified)
                queries.append(response_text)

        # Final answer
        final_context = "\n\n".join(search_results[:5])
        final_prompt = f"""Using this context, provide a comprehensive answer to: {question}

Context:
{final_context}

Answer:"""

        final_response = self.llm.messages.create(
            model="claude-3-sonnet-20240229",
            max_tokens=1024,
            messages=[{"role": "user", "content": final_prompt}]
        )

        return final_response.content[0].text
```

1.5 Examples

1.5.1 Example 1: Complete RAG Implementation

```python
import os
from pathlib import Path
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Pinecone
from langchain.chains import RetrievalQA
from langchain_community.llms import Anthropic

def setup_rag_pipeline(pdf_path: str, index_name: str):
    """Complete RAG setup"""

    # 1. Load documents
    loader = PyPDFLoader(pdf_path)
    documents = loader.load()

    # 2. Split into chunks
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=512,
        chunk_overlap=50
    )
    splits = splitter.split_documents(documents)

    # 3. Embed
    embeddings = HuggingFaceEmbeddings(
        model_name="all-MiniLM-L6-v2"
    )

    # 4. Store in vector DB
    vector_store = Pinecone.from_documents(
        splits,
        embeddings,
        index_name=index_name
    )

    # 5. Create QA chain
    llm = Anthropic()
    qa = RetrievalQA.from_chain_type(
        llm=llm,
        chain_type="stuff",
        retriever=vector_store.as_retriever(search_kwargs={"k": 5})
    )

    return qa

# Usage
qa_pipeline = setup_rag_pipeline("document.pdf", "my-index")
answer = qa_pipeline.run("What is the main topic?")
print(answer)
```

1.5.2 Example 2: Metadata-Aware Chunking

```python
class MetadataChunker:
    def chunk_with_metadata(self, documents: list, chunk_size: int = 512):
        """Preserve document metadata through chunking"""

        chunked_docs = []

        for doc in documents:
            text = doc["content"]
            metadata = doc.get("metadata", {})

            # Split text
            for i in range(0, len(text), chunk_size):
                chunk = text[i:i + chunk_size]

                chunked_docs.append({
                    "content": chunk,
                    "metadata": {
                        **metadata,
                        "chunk_index": i // chunk_size,
                        "source_doc": doc.get("id")
                    }
                })

        return chunked_docs

# Usage
chunker = MetadataChunker()
docs = [
    {
        "id": "doc1",
        "content": "Long text...",
        "metadata": {"source": "manual.pdf", "date": "2024-01-01"}
    }
]
chunks = chunker.chunk_with_metadata(docs)
```

1.5.3 Example 3: RAG Evaluation

```python
from datasets import load_dataset

def evaluate_rag(rag_pipeline, test_questions: list) -> dict:
    """Evaluate RAG pipeline quality"""

    scores = {
        "retrieval_precision": [],
        "answer_relevance": []
    }

    for question, expected_answer in test_questions:
        # Get answer
        answer = rag_pipeline.run(question)

        # Evaluate (simplified - use BERT scores in practice)
        from sentence_transformers import util

        similarity = util.pytorch_cos_sim(
            embedding_model.encode(answer),
            embedding_model.encode(expected_answer)
        )

        scores["answer_relevance"].append(float(similarity))

    # Average scores
    avg_relevance = sum(scores["answer_relevance"]) / len(scores["answer_relevance"])

    return {
        "average_relevance": avg_relevance,
        "sample_answers": [
            rag_pipeline.run(q) for q, _ in test_questions[:3]
        ]
    }
```

1.6 Tools & Ecosystem

Tool	Purpose	URL
LangChain	RAG orchestration	https://docs.langchain.com
LlamaIndex	Document indexing	https://gpt-index.readthedocs.io
Haystack	End-to-end RAG	https://docs.deepset.ai
Ragas	RAG evaluation	https://github.com/explodinggradients/ragas
Vanna	SQL RAG	https://github.com/vanna-ai/vanna

Navigation: Back to Index | Previous: Vector Databases | Next: Prompt Engineering Standards